☕ JAVA COLLECTION FRAMEWORK

Complete Notes + Data Types Interview Guide

From Scratch | Simple Language | Interview-Ready

—— WHAT YOU WILL LEARN ——

📦 Collection Framework — What, Why, When

📋 LIST → ArrayList, LinkedList, Vector, Stack

🔵 SET → HashSet, LinkedHashSet, TreeSet

🗺️ MAP → HashMap, LinkedHashMap, TreeMap, Hashtable

🔄 Queue & Deque → PriorityQueue, ArrayDeque

⚙️ Internal Working of Each Structure

🆚 All vs All Comparison Tables

🎯 50+ Interview Questions & Answers

📊 Data Types — All Differences for Interviews

📦 SECTION 1 — Collection Framework Overview

🔹 What is the Collection Framework?

The Java Collection Framework (JCF) is a unified set of classes and interfaces that provide ready-made data structures to store, organise, and manipulate groups of objects. Think of it as a toolkit where each tool (data structure) is designed for a specific job.

Before collections existed in Java (before JDK 1.2), developers had to write their own data structures from scratch — messy, error-prone, and inconsistent. The Collection Framework solved all of this.

🔹 Why Do We Need Collections?

- Arrays are fixed in size — you cannot resize them. Collections are dynamic.
- Collections have built-in methods: sort, search, add, remove, contains, etc.
- Collections handle memory management automatically.
- Collections follow standard interfaces — code is interchangeable and consistent.
- Different problems need different data structures — collections provide them all.

🔹 Complete Hierarchy — The Big Picture

This is the most important thing to visualise before learning individual collections:

```
java.lang.Iterable
  └── java.util.Collection
        ├── List        (ordered, allows duplicates)
        │     ├── ArrayList
        │     ├── LinkedList
        │     ├── Vector
        │     │     └── Stack
        │     └── CopyOnWriteArrayList  [concurrent]
        │
        ├── Set         (no duplicates)
        │     ├── HashSet
        │     │     └── LinkedHashSet
        │     └── SortedSet
        │           └── NavigableSet
        │                 └── TreeSet
        │
        └── Queue       (FIFO by default)
              ├── PriorityQueue
              └── Deque (double-ended queue)
                    ├── ArrayDeque
                    └── LinkedList  [also implements Deque]

java.util.Map  (separate hierarchy — NOT part of Collection)
  ├── HashMap
  │     └── LinkedHashMap
  ├── SortedMap
  │     └── NavigableMap
  │           └── TreeMap
  ├── Hashtable  [legacy]
```

```
│       └── Properties
└── ConcurrentHashMap  [concurrent]
```

🔹 Key Interfaces Explained in Simple Words

Interface	Simple Meaning	Allows Duplicates?	Ordered?	Key Implementations
Collection	Root interface for all collections	Depends	Depends	List, Set, Queue
List	An ordered sequence (like a numbered list)	Yes	Yes (by index)	ArrayList, LinkedList, Vector
Set	A group with NO repeated elements	No	No (mostly)	HashSet, LinkedHashSet, TreeSet
Queue	A waiting line — first in, first out	Yes	Yes (FIFO)	PriorityQueue, ArrayDeque
Deque	Queue from both ends (front+back)	Yes	Yes	ArrayDeque, LinkedList
Map	Key-value pairs like a dictionary	Keys: No, Values: Yes	No (mostly)	HashMap, TreeMap, LinkedHashMap
SortedSet	A Set that always stays sorted	No	Sorted order	TreeSet
SortedMap	A Map with keys always sorted	Keys: No	Sorted by key	TreeMap

🎯 Interview Questions — Overview

❓ Q: What is the Java Collection Framework?

✅ A: It is a unified architecture in java.util that provides interfaces (List, Set, Map, Queue), their implementations (ArrayList, HashMap, etc.), and algorithms (Collections.sort) to store and manipulate groups of objects. Available since JDK 1.2.

❓ Q: What is the difference between Collection (interface) and Collections (class)?

✅ A: Collection is an interface — the root of the collection hierarchy. Collections (with an "s") is a utility class that provides static methods like Collections.sort(), Collections.reverse(), Collections.shuffle(), Collections.min(), Collections.max().

❓ Q: Why does Map not extend Collection?

✅ A: Because Map stores key-value pairs, whereas Collection stores single elements. Their data models are fundamentally different — Map cannot satisfy the Collection contract (which has methods like add(element), iterator() returning single elements). They are parallel hierarchies.

📋 SECTION 2 — LIST Interface & Implementations

🔹 What is List?

List is an ORDERED collection that allows DUPLICATE elements. Each element has a position (index) starting from 0. You can access any element directly by its index.

Think of it like a numbered railway coach — each seat has a number, order is maintained, and two passengers can have the same name (duplicates allowed).

2.1 — ArrayList

🔹 What is ArrayList?

ArrayList is the most commonly used collection. Internally, it uses a DYNAMIC ARRAY. When the array fills up, it automatically grows by creating a new, larger array and copying elements.

Internal Working of ArrayList

1. Default initial capacity = 10 slots are created in memory.
2. When you add elements, they fill slots one by one.
3. When 10 slots are full and you add the 11th element:
4. Java creates a new array of size: old size × 1.5 (so 10 → 15).
5. All existing elements are COPIED to the new array.
6. The 11th element is then added. Old array is garbage collected.

> 💡 **NOTE:** This resizing (copying) takes O(n) time, which is why adding to a full ArrayList is temporarily slow. On average, add() is O(1) — called "amortized constant time".

ArrayList — Full Code with All Operations

```
import java.util.*;

public class ArrayListDemo {
    public static void main(String[] args) {

        // 1. Create ArrayList
        ArrayList<String> cities = new ArrayList<>();         // empty, capacity 10
        ArrayList<Integer> nums  = new ArrayList<>(20);      // custom initial capacity
        ArrayList<String> copy   = new ArrayList<>(cities);  // copy constructor

        // 2. Add elements
        cities.add("Mumbai");           // add at end → O(1) amortized
        cities.add("Delhi");
        cities.add("Pune");
        cities.add(1, "Chennai");       // add at index 1 → O(n) [shifts others]
        System.out.println(cities);     // [Mumbai, Chennai, Delhi, Pune]

        // 3. Access elements
```

```java
        System.out.println(cities.get(0));        // Mumbai → O(1)
        System.out.println(cities.size());        // 4
        System.out.println(cities.indexOf("Delhi")); // 2
        System.out.println(cities.lastIndexOf("Delhi")); // 2

        // 4. Update elements
        cities.set(0, "Kolkata");       // replace index 0 → O(1)
        System.out.println(cities.get(0));  // Kolkata

        // 5. Remove elements
        cities.remove("Pune");          // remove by value → O(n)
        cities.remove(0);               // remove by index → O(n) [shifts]
        System.out.println(cities);     // [Chennai, Delhi]

        // 6. Search
        System.out.println(cities.contains("Delhi"));   // true → O(n)
        System.out.println(cities.isEmpty());           // false

        // 7. Iterating — 4 ways
        // a) for-each (most common)
        for (String c : cities) System.out.println(c);

        // b) index-based for loop
        for (int i = 0; i < cities.size(); i++) System.out.println(cities.get(i));

        // c) Iterator
        Iterator<String> it = cities.iterator();
        while (it.hasNext()) System.out.println(it.next());

        // d) forEach with Lambda (Java 8+)
        cities.forEach(c -> System.out.println(c));

        // 8. Sort
        ArrayList<Integer> scores = new ArrayList<>(Arrays.asList(50,20,80,10,60));
        Collections.sort(scores);                   // [10, 20, 50, 60, 80]
        Collections.sort(scores, Collections.reverseOrder()); // [80, 60, 50, 20, 10]

        // 9. Convert to Array
        String[] arr = cities.toArray(new String[0]);

        // 10. subList
        List<Integer> sub = scores.subList(1, 3);  // indices 1 to 2

        // 11. addAll
        cities.addAll(Arrays.asList("Jaipur", "Surat"));

        // 12. Clear
        cities.clear();
        System.out.println(cities.isEmpty()); // true
    }
}
```

ArrayList Time Complexity

Operation	Time Complexity	Why?
get(index)	O(1)	Direct index access in array — fastest
add(element)	O(1) amortized	Usually appends at end. Occasionally O(n) for resize+copy
add(index, elem)	O(n)	Must shift all elements after index to the right

Operation	Time Complexity	Why?
remove(index)	O(n)	Must shift all elements after index to the left
remove(object)	O(n)	Must scan to find element, then shift
contains()	O(n)	Linear scan through entire array
indexOf()	O(n)	Linear scan
size()	O(1)	Size is stored as a variable
set(index, val)	O(1)	Direct index write
sort()	O(n log n)	Uses TimSort algorithm internally

2.2 — LinkedList

◆ What is LinkedList?

LinkedList uses a DOUBLY LINKED LIST internally. Each element is a "Node" object that contains the data, a pointer to the NEXT node, and a pointer to the PREVIOUS node.

Think of it like a chain of paper clips — each clip is connected to the one before and after it. To insert or remove, you just re-link the clips. But to find clip number 50, you must count one by one from the start.

```
Node structure inside LinkedList:

  [prev|data|next] <--> [prev|data|next] <--> [prev|data|next]
   Node 1               Node 2               Node 3

  head = first node     tail = last node
```

LinkedList — Full Code with All Operations

```
import java.util.*;

public class LinkedListDemo {
    public static void main(String[] args) {

        LinkedList<String> ll = new LinkedList<>();

        // Add operations (LinkedList is also a Deque)
        ll.add("B");            // add at end
        ll.addFirst("A");       // add at beginning → O(1)
        ll.addLast("C");        // add at end → O(1)
        ll.add(1, "AB");        // add at index 1 → O(n) to traverse
        System.out.println(ll); // [A, AB, B, C]

        // Access
        System.out.println(ll.get(2));     // B   → O(n) — must traverse!
        System.out.println(ll.getFirst()); // A   → O(1)
        System.out.println(ll.getLast());  // C   → O(1)
        System.out.println(ll.peek());     // A   → O(1) [peek = getFirst, no exception]
        System.out.println(ll.peekFirst());// A
        System.out.println(ll.peekLast()); // C

        // Remove operations
```

```
        ll.removeFirst();       // removes A → O(1)
        ll.removeLast();        // removes C → O(1)
        ll.remove("B");         // removes by value → O(n)
        System.out.println(ll); // [AB]

        // Queue operations (FIFO)
        ll.offer("X");      // add to end (queue-style)
        ll.poll();          // remove from front (returns null if empty)

        // Stack operations (LIFO)
        ll.push("Y");       // add to front (stack-style)
        ll.pop();           // remove from front

        // Iterate
        for (String s : ll) System.out.println(s);

        // Descending iterator
        Iterator<String> desc = ll.descendingIterator();
        while (desc.hasNext()) System.out.println(desc.next());
    }
}
```

LinkedList Time Complexity

Operation	Time Complexity	Why?
addFirst() / addLast()	O(1)	Just change head or tail pointer
add(index, elem)	O(n)	Must traverse to that index first
get(index)	O(n)	No direct access — must traverse node by node
getFirst() / getLast()	O(1)	Head and tail are directly accessible
removeFirst() / removeLast()	O(1)	Just update head/tail pointer
remove(index)	O(n)	Must traverse to find the node
contains()	O(n)	Linear scan through nodes
size()	O(1)	Size is tracked as a variable

2.3 — Vector (Legacy)

◆ What is Vector?

Vector is similar to ArrayList — it uses a dynamic array internally. The key difference is that all Vector methods are SYNCHRONIZED, making it thread-safe. However, this comes at a performance cost.

Vector doubles its size when full (ArrayList grows by 50%). Vector is considered LEGACY — in modern Java, prefer ArrayList (single-threaded) or CopyOnWriteArrayList / Collections.synchronizedList (multi-threaded).

```
import java.util.*;

Vector<String> v = new Vector<>();         // default capacity: 10, doubles when full
```

```
Vector<String> v2 = new Vector<>(20);      // initial capacity 20
Vector<String> v3 = new Vector<>(10, 5);   // capacity 10, grows by 5 each time

v.add("Apple");
v.add("Banana");
v.addElement("Cherry");    // legacy method, same as add()

System.out.println(v.firstElement()); // Apple
System.out.println(v.lastElement());  // Cherry
System.out.println(v.capacity());     // 10 (internal array size)
System.out.println(v.size());         // 3  (actual elements)

v.removeElement("Banana");
v.insertElementAt("Date", 1);

Enumeration<String> e = v.elements(); // legacy iterator
while (e.hasMoreElements()) System.out.println(e.nextElement());
```

⚠️ **WARNING:** Do NOT use Vector in new code. It is a legacy class. All methods are synchronized, which causes unnecessary locking overhead even in single-threaded code.

2.4 — Stack (Legacy)

🔹 What is Stack?

Stack extends Vector and represents a Last-In-First-Out (LIFO) data structure. Think of a stack of plates — you put on top and take from top.

```
import java.util.*;

Stack<Integer> stack = new Stack<>();

stack.push(10);   // push to top
stack.push(20);
stack.push(30);
System.out.println(stack);         // [10, 20, 30]

System.out.println(stack.peek());  // 30 — see top, don't remove
System.out.println(stack.pop());   // 30 — remove and return top
System.out.println(stack.pop());   // 20
System.out.println(stack.isEmpty()); // false

System.out.println(stack.search(10)); // 1 (position from top, 1-based)

// PREFERRED ALTERNATIVE: ArrayDeque
Deque<Integer> dequeStack = new ArrayDeque<>();
dequeStack.push(10);
dequeStack.push(20);
System.out.println(dequeStack.peek()); // 20
dequeStack.pop();
```

🎯 **INTERVIEW TIP:** In interviews, if asked about Stack, mention that java.util.Stack is legacy. The PREFERRED modern alternative is ArrayDeque used as a stack (push/pop/peek). ArrayDeque is faster as it is not synchronized.

🎯 Interview Questions — LIST

❓ Q: What is the difference between ArrayList and LinkedList?

✅ A: ArrayList uses a dynamic array — O(1) for get() by index, O(n) for insert/delete in middle. LinkedList uses doubly linked list — O(n) for get(), but O(1) for addFirst/addLast. Use ArrayList when you read a lot; use LinkedList when you add/remove at ends frequently.

❓ Q: What is the difference between ArrayList and Vector?

✅ A: ArrayList is NOT synchronized (not thread-safe), faster. Vector IS synchronized (thread-safe), slower. ArrayList grows by 50%, Vector doubles. ArrayList is modern (JDK 1.2+), Vector is legacy (JDK 1.0). For thread-safety, prefer Collections.synchronizedList(new ArrayList()) over Vector.

❓ Q: How does ArrayList resize internally?

✅ A: When capacity is full, ArrayList creates a new array of 1.5x the old size, copies all elements (O(n)), then adds the new element. This is why occasional add() is O(n). The amortized cost across many adds is still O(1).

❓ Q: Can ArrayList store null values?

✅ A: Yes. ArrayList allows multiple null values. list.add(null) is valid. However, calling methods on null elements will cause NullPointerException.

❓ Q: What is fail-fast iterator?

✅ A: ArrayList's iterator is fail-fast — if the list is structurally modified after creating the iterator (except through iterator.remove()), it throws ConcurrentModificationException. This is detected via a modCount variable.

❓ Q: What happens when you call list.remove(1) vs list.remove(Integer.valueOf(1))?

✅ A: list.remove(1) removes the element at INDEX 1. list.remove(Integer.valueOf(1)) removes the OBJECT with value 1. This is a classic interview trick — auto-unboxing means the int literal goes to the index-based overload!

🔵 SECTION 3 — SET Interface & Implementations

🔹 What is Set?

Set is a collection that does NOT allow duplicate elements. If you try to add a duplicate, the add() method returns false and the element is simply ignored — no error, no exception.

Think of it like a guest list for an exclusive party — each person can appear only once. Sets are ideal when uniqueness matters.

3.1 — HashSet

🔹 What is HashSet?

HashSet is the most commonly used Set. It uses a HashMap internally — each element you add becomes a KEY in the internal HashMap (with a dummy value PRESENT).

Because it uses HashMap, insertion order is NOT guaranteed. Elements may appear in any order when you iterate.

HashSet Internal Working — Step by Step

7. When you add("Apple") to HashSet:
8. Java calls "Apple".hashCode() — gets a number like 63476345.
9. This hash is processed: index = hash & (n-1), where n = array size (default 16).
10. "Apple" is stored at that index in the internal HashMap as a key.
11. When you add("Apple") again — same hash, same index, key already exists → IGNORED.
12. When two different strings have the same index (hash collision) → stored in a linked list at that bucket.
13. Java 8+: if a bucket has more than 8 entries → linked list converts to Red-Black Tree → O(log n).

💡 **NOTE:** HashSet allows ONE null element. null is stored at bucket index 0 (hashCode of null is 0).

HashSet — Full Code

```
import java.util.*;

public class HashSetDemo {
    public static void main(String[] args) {

        HashSet<String> fruits = new HashSet<>();

        // add() returns boolean — true if added, false if duplicate
        System.out.println(fruits.add("Mango"));    // true
        System.out.println(fruits.add("Apple"));    // true
        System.out.println(fruits.add("Banana"));   // true
        System.out.println(fruits.add("Apple"));    // false — duplicate ignored
        System.out.println(fruits.add(null));       // true — 1 null allowed
```

```
        System.out.println(fruits);  // order NOT guaranteed, e.g. [null, Banana, Apple,
Mango]
        System.out.println(fruits.size());          // 4

        // Search — O(1) average
        System.out.println(fruits.contains("Mango"));  // true
        System.out.println(fruits.contains("Grapes")); // false

        // Remove — O(1) average
        fruits.remove("Banana");
        System.out.println(fruits); // [null, Apple, Mango]

        // Iterate (no guaranteed order)
        for (String f : fruits) System.out.println(f);

        // Set operations using addAll, retainAll, removeAll
        HashSet<Integer> set1 = new HashSet<>(Arrays.asList(1,2,3,4,5));
        HashSet<Integer> set2 = new HashSet<>(Arrays.asList(3,4,5,6,7));

        // Union (A ∪ B)
        HashSet<Integer> union = new HashSet<>(set1);
        union.addAll(set2);
        System.out.println("Union: " + union);      // [1,2,3,4,5,6,7]

        // Intersection (A ∩ B)
        HashSet<Integer> intersection = new HashSet<>(set1);
        intersection.retainAll(set2);
        System.out.println("Intersection: " + intersection); // [3,4,5]

        // Difference (A - B)
        HashSet<Integer> diff = new HashSet<>(set1);
        diff.removeAll(set2);
        System.out.println("Difference: " + diff);  // [1,2]
    }
}
```

3.2 — LinkedHashSet

◆ What is LinkedHashSet?

LinkedHashSet extends HashSet and uses a LinkedHashMap internally. It maintains INSERTION ORDER — elements come out in the same order you put them in. No duplicates.

Think of it as a VIP guest list where the arrival order is remembered and each name appears only once.

```
import java.util.*;

LinkedHashSet<String> lhs = new LinkedHashSet<>();

lhs.add("C");
lhs.add("A");
lhs.add("B");
lhs.add("A");   // duplicate — ignored, order preserved

System.out.println(lhs);  // [C, A, B]  — insertion order maintained!

// Good use case: remove duplicates from a list but preserve order
List<String> withDups = Arrays.asList("X","Y","X","Z","Y","W");
```

```
LinkedHashSet<String> unique = new LinkedHashSet<>(withDups);
System.out.println(unique);  // [X, Y, Z, W]
```

3.3 — TreeSet

◆ What is TreeSet?

TreeSet implements SortedSet and NavigableSet. It stores elements in SORTED ORDER (natural ordering by default). Internally uses a Red-Black Tree (a self-balancing binary search tree).

Think of it like an alphabetical dictionary — every word is always stored in sorted order, no duplicates.

```
import java.util.*;

public class TreeSetDemo {
    public static void main(String[] args) {

        TreeSet<Integer> ts = new TreeSet<>();
        ts.add(50);
        ts.add(10);
        ts.add(80);
        ts.add(30);
        ts.add(10);  // duplicate — ignored

        System.out.println(ts);         // [10, 30, 50, 80] — sorted!

        // Navigation methods (unique to TreeSet)
        System.out.println(ts.first());          // 10 — smallest
        System.out.println(ts.last());           // 80 — largest
        System.out.println(ts.floor(35));        // 30 — largest element <= 35
        System.out.println(ts.ceiling(35));      // 50 — smallest element >= 35
        System.out.println(ts.lower(30));        // 10 — largest element strictly < 30
        System.out.println(ts.higher(30));       // 50 — smallest element strictly > 30

        // Sub-sets
        System.out.println(ts.headSet(50));      // [10, 30]    — elements < 50
        System.out.println(ts.tailSet(30));      // [30, 50, 80] — elements >= 30
        System.out.println(ts.subSet(10, 50));   // [10, 30]    — 10 <= x < 50

        // Reverse order
        TreeSet<Integer> desc = new TreeSet<>(Collections.reverseOrder());
        desc.addAll(ts);
        System.out.println(desc); // [80, 50, 30, 10]

        // Custom sorting with Comparator
        TreeSet<String> byLength = new TreeSet<>(

Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder())
        );
        byLength.add("Banana");
        byLength.add("Fig");
        byLength.add("Apple");
        byLength.add("Ki");
        System.out.println(byLength); // [Ki, Fig, Apple, Banana]
    }
}
```

⚠️ **WARNING:** TreeSet does NOT allow null. Adding null throws NullPointerException because null cannot be compared for sorting. HashSet and LinkedHashSet allow one null.

🎯 All Set Comparison Table

Feature	HashSet	LinkedHashSet	TreeSet
Internal Structure	HashMap	LinkedHashMap	Red-Black Tree
Order of elements	No order (random)	Insertion order	Sorted (natural / Comparator)
Allows duplicates?	No	No	No
Allows null?	Yes (1 null)	Yes (1 null)	NO — NullPointerException
Thread safe?	No	No	No
add() / remove() / contains()	O(1) average	O(1) average	O(log n)
Memory usage	Less	More (doubly linked list)	More (tree nodes)
Navigation methods	None	None	first(), last(), floor(), ceiling()
When to use	Fastest — just unique elements	Unique + preserve insertion order	Unique + always sorted
Real-world example	Remove duplicates from data	Ordered unique tags/categories	Leaderboard, sorted words, range queries

🎯 Interview Questions — SET

❓ Q: How does HashSet prevent duplicates?

✅ A: HashSet uses HashMap internally. When you add(element), it calls element.hashCode() to find the bucket, then checks with equals() if an identical key exists. If yes, the element is not inserted. This is why you must override both hashCode() and equals() in custom objects stored in HashSet.

❓ Q: Why must we override both hashCode() and equals() for custom objects in HashSet?

✅ A: If two objects are equal (equals() returns true), their hashCode() MUST also be equal — otherwise HashSet will store them as separate entries in different buckets. The contract: equals implies same hashCode. Breaking this causes duplicate "equal" objects in HashSet.

❓ Q: What is the difference between HashSet and TreeSet?

✅ A: HashSet: O(1) operations, no order, allows null, backed by HashMap. TreeSet: O(log n) operations, always sorted, no null, backed by Red-Black Tree. Choose TreeSet when you need sorted access; HashSet for maximum speed.

🗺️ SECTION 4 — MAP Interface & Implementations

🔹 What is Map?

Map stores data as KEY-VALUE pairs. Each key is unique — if you put the same key twice, the second value OVERWRITES the first. Values can be duplicate.

Think of a Map like a real dictionary: each word (key) is unique, and each word has a meaning (value). Or like a phone book: each name maps to a phone number.

```
Map<String, Integer> scores = new HashMap<>();
scores.put("Alice", 90);
scores.put("Bob",   80);
scores.put("Alice", 95);  // duplicate key — 90 is OVERWRITTEN by 95
System.out.println(scores); // {Bob=80, Alice=95}
```

4.1 — HashMap

🔹 What is HashMap?

HashMap is the most widely used Map implementation. It uses an array of buckets + linked list (+ tree in Java 8+) internally.

HashMap Internal Working — Deep Dive (Interview Gold!)

```
INTERNAL STRUCTURE:

  Array of Buckets (default size 16):
  [0] → null
  [1] → Node{key="Dog",  value=3, next=null}
  [2] → Node{key="Cat",  value=1, next=→ Node{key="Bat", value=2}}  ← collision!
  [3] → null
  ...
  [15]→ null

Each "Node" stores: key, value, hash, next
```

put(key, value) — Step by Step:

14. Compute hashCode() of the key.
15. Apply hash spreading: (h = key.hashCode()) ^ (h >>> 16) — reduces collision.
16. Find bucket index: index = hash & (capacity - 1).
17. If bucket is empty: create a new Node and place it there.
18. If bucket has nodes (collision):
19. Walk the linked list. Compare each node using hashCode() AND equals().
20. If matching key found: UPDATE the value.
21. If no match found: APPEND new node at end of list (Java 8+: at head before Java 8).
22. If a bucket's list size exceeds 8 AND total capacity >= 64: convert to Red-Black Tree.
23. If size > (capacity × loadFactor): REHASH — create new array (2x size), re-insert all.

💡 **NOTE:** Default initial capacity = 16. Default load factor = 0.75. Rehashing happens when size > 12 (16 × 0.75).

HashMap — Complete Code

```
import java.util.*;

public class HashMapDemo {
    public static void main(String[] args) {

        // 1. Create HashMap
        HashMap<String, Integer> map = new HashMap<>();
        HashMap<String, Integer> m2  = new HashMap<>(32);          // initial capacity 32
        HashMap<String, Integer> m3  = new HashMap<>(32, 0.5f);   // capacity 32,
loadFactor 0.5

        // 2. put() — add / update key-value pairs
        map.put("Alice", 90);
        map.put("Bob",   75);
        map.put("Carol", 88);
        map.put("Bob",   80);   // overwrites 75
        map.put(null, 0);       // 1 null key allowed
        System.out.println(map);  // {null=0, Bob=80, Carol=88, Alice=90} (order varies)

        // 3. get() — retrieve value by key
        System.out.println(map.get("Alice"));      // 90
        System.out.println(map.get("Zara"));       // null (key not found)

        // 4. getOrDefault() — safe get with fallback
        System.out.println(map.getOrDefault("Zara", -1)); // -1

        // 5. containsKey / containsValue
        System.out.println(map.containsKey("Carol"));   // true
        System.out.println(map.containsValue(99));      // false

        // 6. remove()
        map.remove("Carol");              // remove by key
        map.remove("Bob", 80);            // remove only if value matches → Java 8+

        // 7. putIfAbsent() — only put if key is missing
        map.putIfAbsent("Dave", 70);
        map.putIfAbsent("Alice", 100);   // Alice already exists — ignored

        // 8. compute / merge (Java 8+)
        map.compute("Alice", (k, v) -> v == null ? 1 : v + 1); // increment
        map.merge("Alice", 5, Integer::sum);   // Alice = old + 5

        // 9. Iterating — multiple ways
        // a) entrySet (most efficient — access key+value together)
        for (Map.Entry<String, Integer> e : map.entrySet()) {
            System.out.println(e.getKey() + " => " + e.getValue());
        }

        // b) keySet
        for (String key : map.keySet()) {
            System.out.println(key + " -> " + map.get(key));
        }

        // c) values only
```

```
        for (int val : map.values()) System.out.println(val);

        // d) forEach with Lambda (Java 8+)
        map.forEach((k, v) -> System.out.println(k + ": " + v));

        // 10. size, isEmpty, clear
        System.out.println(map.size());
        System.out.println(map.isEmpty());
        map.clear();
    }
}
```

4.2 — LinkedHashMap

🔹 What is LinkedHashMap?

LinkedHashMap extends HashMap and adds a doubly linked list that connects all entries in INSERTION ORDER (or access order if configured). So you get O(1) average performance PLUS ordered iteration.

```
import java.util.*;

LinkedHashMap<String, Integer> lhm = new LinkedHashMap<>();
lhm.put("Banana", 3);
lhm.put("Apple",  5);
lhm.put("Cherry", 1);

System.out.println(lhm); // {Banana=3, Apple=5, Cherry=1} — insertion order kept!

// Access-order mode (true = access order, false = insertion order)
LinkedHashMap<String, Integer> lru = new LinkedHashMap<>(16, 0.75f, true);
lru.put("A", 1); lru.put("B", 2); lru.put("C", 3);
lru.get("A");   // access A — moves A to end
System.out.println(lru); // {B=2, C=3, A=1} — A moved to end (recently accessed)

// LRU Cache implementation using LinkedHashMap!
LinkedHashMap<Integer, String> lruCache = new LinkedHashMap<Integer, String>(
    16, 0.75f, true) {
    @Override
    protected boolean removeEldestEntry(Map.Entry<Integer,String> eldest) {
        return size() > 3;  // evict oldest when size exceeds 3
    }
};
```

🎯 **INTERVIEW TIP:** LRU Cache using LinkedHashMap is a VERY popular interview question at top companies. Remember: LinkedHashMap(capacity, loadFactor, accessOrder=true) + override removeEldestEntry().

4.3 — TreeMap

🔹 What is TreeMap?

TreeMap implements SortedMap and NavigableMap. Keys are always stored in SORTED ORDER (natural order or custom Comparator). Internally uses a Red-Black Tree.

```java
import java.util.*;

TreeMap<String, Integer> tm = new TreeMap<>();
tm.put("Banana", 3);
tm.put("Apple",  5);
tm.put("Cherry", 1);
tm.put("Date",   7);

System.out.println(tm);  // {Apple=5, Banana=3, Cherry=1, Date=7} — sorted by key!

// Navigation
System.out.println(tm.firstKey());          // Apple
System.out.println(tm.lastKey());           // Date
System.out.println(tm.floorKey("Blueberry")); // Banana (largest key <= "Blueberry")
System.out.println(tm.ceilingKey("C"));       // Cherry (smallest key >= "C")
System.out.println(tm.lowerKey("Cherry"));    // Banana (strictly less than "Cherry")
System.out.println(tm.higherKey("Banana"));   // Cherry (strictly greater than "Banana")

// Sub-maps
System.out.println(tm.headMap("Cherry"));     // {Apple=5, Banana=3} — keys < "Cherry"
System.out.println(tm.tailMap("Cherry"));     // {Cherry=1, Date=7}  — keys >= "Cherry"
System.out.println(tm.subMap("B", "D"));      // {Banana=3, Cherry=1} — "B" <= key < "D"

// Descending order
System.out.println(tm.descendingKeySet()); // [Date, Cherry, Banana, Apple]

// Custom comparator (reverse order)
TreeMap<Integer, String> rev = new TreeMap<>(Collections.reverseOrder());
rev.put(3,"C"); rev.put(1,"A"); rev.put(2,"B");
System.out.println(rev); // {3=C, 2=B, 1=A}
```

4.4 — Hashtable (Legacy)

Hashtable is the legacy thread-safe map. All methods are synchronized. Does NOT allow null keys or null values. Considered obsolete — use ConcurrentHashMap for thread safety instead.

```java
Hashtable<String, Integer> ht = new Hashtable<>();
ht.put("A", 1);
ht.put("B", 2);
// ht.put(null, 3);  // NullPointerException!
// ht.put("C", null); // NullPointerException!
System.out.println(ht); // {B=2, A=1}  — no guaranteed order
```

🎯 All Map Comparison Table

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable
Internal Structure	Array+List/Tree	Array+List/Tree+LinkedList	Red-Black Tree	Array+List
Key Order	No order	Insertion order	Sorted (natural/Comparator)	No order
Null Keys	1 null key OK	1 null key OK	NO null keys	NO null keys
Null Values	Yes	Yes	Yes	NO null values
Thread Safe?	No	No	No	Yes (all synchronized)

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable
put/get/remove	O(1) average	O(1) average	O(log n)	O(1) average
Best Use	General purpose	Ordered + fast	Sorted key access	Legacy only
Introduced	Java 1.2	Java 1.4	Java 1.2	Java 1.0 (legacy)

🎯 Interview Questions — MAP

❓ Q: What is the time complexity of HashMap get() and put()?

✅ A: Average case: O(1). Worst case: O(n) if all keys hash to the same bucket (all in one linked list). In Java 8+, worst case is O(log n) because long chains convert to Red-Black Trees.

❓ Q: What happens if two keys have the same hashCode?

✅ A: This is called a hash collision. Both entries are stored in the same bucket as a linked list (Java 8+: tree if >8 entries). On get(), Java traverses the list and uses equals() to find the exact key.

❓ Q: Why should we override hashCode() and equals() in HashMap keys?

✅ A: If two objects are logically equal (equals() returns true) but have different hashCodes, HashMap will put them in different buckets — so get() with an "equal" key will return null. The contract: if a.equals(b) then a.hashCode() == b.hashCode().

❓ Q: What is the difference between HashMap and ConcurrentHashMap?

✅ A: HashMap is not thread-safe. ConcurrentHashMap is thread-safe using segment-level locking (Java 7) or CAS + synchronized buckets (Java 8+). ConcurrentHashMap is much faster than Hashtable because it does not lock the entire map — only the affected bucket.

❓ Q: What is the difference between HashMap and LinkedHashMap?

✅ A: HashMap has no order. LinkedHashMap maintains insertion order (or access order) via an extra doubly linked list. LinkedHashMap uses slightly more memory but is still O(1) for basic operations.

🔄 SECTION 5 — Queue & Deque

🔹 What is Queue?

Queue is a FIFO (First-In-First-Out) data structure. The element added first is removed first — like a ticket queue at a bus stop. Java Queue interface extends Collection.

5.1 — PriorityQueue

🔹 What is PriorityQueue?

PriorityQueue is a Queue where elements are processed based on PRIORITY, not insertion order. By default, the smallest element has highest priority (min-heap). You can change this with a Comparator.

```
import java.util.*;

public class PriorityQueueDemo {
    public static void main(String[] args) {

        // Min-heap (default) — smallest element polled first
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.offer(50);
        pq.offer(10);
        pq.offer(30);
        pq.offer(20);

        System.out.println(pq.peek());   // 10 — smallest, not removed
        System.out.println(pq.poll());   // 10 — smallest, removed
        System.out.println(pq.poll());   // 20
        System.out.println(pq.poll());   // 30

        // Max-heap — largest element polled first
        PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder());
        maxPQ.offer(50); maxPQ.offer(10); maxPQ.offer(30);
        System.out.println(maxPQ.poll()); // 50 — largest first

        // PriorityQueue with custom objects
        PriorityQueue<String> byLength = new PriorityQueue<>(
            Comparator.comparingInt(String::length));
        byLength.offer("Banana");
        byLength.offer("Ki");
        byLength.offer("Apple");
        System.out.println(byLength.poll()); // Ki (shortest)
        System.out.println(byLength.poll()); // Apple
    }
}
```

💡 **NOTE:** PriorityQueue internally uses a binary heap array. add()/offer() is O(log n). poll() is O(log n). peek() is O(1). Iteration order is NOT guaranteed to be sorted — only poll() gives sorted order.

5.2 — ArrayDeque

◆ What is ArrayDeque?

ArrayDeque (Array Double-Ended Queue) allows adding and removing elements from BOTH ends. It is faster than LinkedList as a queue/stack and faster than Stack for LIFO operations. It is the PREFERRED Stack and Queue implementation in modern Java.

```
import java.util.*;

ArrayDeque<String> deque = new ArrayDeque<>();

// Adding to both ends
deque.offerFirst("B");   // [B]
deque.offerLast("C");    // [B, C]
deque.offerFirst("A");   // [A, B, C]
deque.addLast("D");      // [A, B, C, D]

// Peek (no removal)
System.out.println(deque.peekFirst()); // A
System.out.println(deque.peekLast());  // D

// Poll (removes and returns)
System.out.println(deque.pollFirst()); // A → [B, C, D]
System.out.println(deque.pollLast());  // D → [B, C]

// Use as Stack (LIFO)
deque.push("X");    // addFirst → [X, B, C]
deque.pop();        // removeFirst → [B, C]

// Use as Queue (FIFO)
deque.offer("E");   // addLast → [B, C, E]
deque.poll();       // removeFirst → [C, E]
```

🎯 **INTERVIEW TIP:** ArrayDeque does NOT allow null elements. It is NOT thread-safe. For thread-safe deque, use LinkedBlockingDeque. Always prefer ArrayDeque over Stack class in interviews!

Feature	ArrayDeque	LinkedList	Stack	PriorityQueue
Internal structure	Resizable circular array	Doubly linked list	Extends Vector (array)	Binary heap array
Null allowed?	NO	Yes	Yes	NO
Access by index?	No	Yes (but O(n))	Yes (but O(n))	No
Thread safe?	No	No	Yes (synchronized)	No
Best use as Stack	YES — fastest	OK	Legacy only	N/A
Best use as Queue	YES — fastest	OK	N/A	N/A
Priority ordering	No — FIFO/LIFO only	No	No	YES — by priority
Time: add/remove ends	O(1) amortized	O(1)	O(1)	O(log n)

🎯 Interview Questions — Queue & Deque

❓ **Q: What is the difference between offer() and add() in Queue?**

✅ A: Both add an element to the queue. The difference: add() throws an exception (IllegalStateException) if the queue is full (in bounded queues). offer() returns false instead of throwing. Prefer offer() for safety.

❓ **Q: What is the difference between poll() and remove() in Queue?**

✅ A: Both remove the head element. remove() throws NoSuchElementException if empty. poll() returns null if empty. Prefer poll() for safety — it never throws.

❓ **Q: What is the difference between peek() and element() in Queue?**

✅ A: Both look at the head without removing. element() throws NoSuchElementException if empty. peek() returns null. Always prefer peek().

⚙️ SECTION 6 — Collections Utility Class & Iterator

6.1 — Collections Utility Class

java.util.Collections provides static helper methods for collection operations. Very frequently tested in interviews!

```
import java.util.*;

List<Integer> list = new ArrayList<>(Arrays.asList(5, 3, 8, 1, 9, 2));

// Sorting
Collections.sort(list);                        // [1, 2, 3, 5, 8, 9] — ascending
Collections.sort(list, Collections.reverseOrder()); // [9, 8, 5, 3, 2, 1]

// Min and Max
System.out.println(Collections.min(list));    // 1
System.out.println(Collections.max(list));    // 9

// Binary Search (list must be sorted first!)
Collections.sort(list);
System.out.println(Collections.binarySearch(list, 5)); // index of 5

// Reverse
Collections.reverse(list);

// Shuffle (random order)
Collections.shuffle(list);
Collections.shuffle(list, new Random(42)); // seeded random

// Frequency — count occurrences
List<String> words = Arrays.asList("hi","bye","hi","ok","hi");
System.out.println(Collections.frequency(words, "hi")); // 3

// Fill — replace all with one value
Collections.fill(list, 0); // [0, 0, 0, 0, 0, 0]

// Copy — destination must have same or larger size
List<Integer> src  = Arrays.asList(1,2,3);
List<Integer> dest = Arrays.asList(0,0,0);
Collections.copy(dest, src);

// Unmodifiable wrapper
List<String> immutable = Collections.unmodifiableList(new
ArrayList<>(Arrays.asList("A","B")));
// immutable.add("C"); // throws UnsupportedOperationException

// Synchronized wrapper (thread-safe)
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
Map<String,Integer> syncMap = Collections.synchronizedMap(new HashMap<>());

// Singleton — immutable single-element collection
Set<String> single = Collections.singleton("only");
List<Integer> singleList = Collections.singletonList(42);

// nCopies
```

```
List<String> repeated = Collections.nCopies(5, "Java");
// ["Java", "Java", "Java", "Java", "Java"]

// disjoint — returns true if no common elements
List<Integer> a = Arrays.asList(1,2,3);
List<Integer> b = Arrays.asList(4,5,6);
System.out.println(Collections.disjoint(a, b)); // true
```

6.2 — Iterator vs ListIterator vs for-each

Feature	for-each loop	Iterator	ListIterator
Direction	Forward only	Forward only	Forward AND Backward
Remove during iter?	NO — ConcurrentModificationException	YES — iterator.remove()	YES — remove() / add() / set()
Add during iter?	No	No	YES — listIterator.add()
Access index?	No	No	YES — nextIndex() / previousIndex()
Works on	Any Iterable	Any Collection	Only List
Syntax	for(T t : col)	col.iterator()	list.listIterator()

```
// Safe removal during iteration — ALWAYS use iterator.remove()
List<Integer> nums = new ArrayList<>(Arrays.asList(1,2,3,4,5,6));
Iterator<Integer> it = nums.iterator();
while (it.hasNext()) {
    int n = it.next();
    if (n % 2 == 0) it.remove();  // removes even numbers safely
}
System.out.println(nums); // [1, 3, 5]

// ListIterator — bidirectional
ListIterator<String> lit = new ArrayList<>(Arrays.asList("A","B","C")).listIterator();
while (lit.hasNext()) {
    String s = lit.next();
    lit.set(s.toLowerCase()); // replace in-place
}
```

🎯 Interview Questions — Collections Utility

❓ **Q: Why do we get ConcurrentModificationException?**

✅ A: When you modify a collection (add/remove) WHILE iterating over it with a for-each or iterator (except via iterator.remove()), the iterator detects the structural change via modCount and throws ConcurrentModificationException. Solution: use iterator.remove() or Java 8 removeIf().

❓ **Q: What is the difference between Iterator and ListIterator?**

✅ A: Iterator works on any Collection, traverses forward only, supports only remove(). ListIterator works only on List, can go forward and backward, supports add(), remove(), and set() during iteration.

🏆 SECTION 7 — MASTER COMPARISON: All Collections

🔹 All Collections at a Glance — The Ultimate Cheat Sheet

Collection	Type	Order	Duplicates?	Null?	Thread-Safe?	Time (add/get/remove)
ArrayList	List	Insertion index	Yes	Yes	No	add:O(1)*, get:O(1), del:O(n)
LinkedList	List/Deque	Insertion	Yes	Yes	No	addFirst/Last:O(1), get:O(n)
Vector	List	Insertion index	Yes	Yes	Yes	add:O(1)*, get:O(1), del:O(n)
Stack	List	LIFO	Yes	Yes	Yes	push/pop:O(1)
HashSet	Set	No order	No	1 null	No	O(1) average
LinkedHashSet	Set	Insertion order	No	1 null	No	O(1) average
TreeSet	Set	Sorted	No	NO	No	O(log n)
PriorityQueue	Queue	Priority order	Yes	NO	No	offer:O(log n),peek:O(1)
ArrayDeque	Deque	Insertion/FIFO/LIFO	Yes	NO	No	add/remove ends:O(1)
HashMap	Map	No order	Values	1 null key	No	O(1) average
LinkedHashMap	Map	Insertion order	Values	1 null key	No	O(1) average
TreeMap	Map	Sorted by key	Values	NO null key	No	O(log n)
Hashtable	Map	No order	Values	NO nulls	Yes	O(1) average

🔹 WHEN TO USE WHICH COLLECTION — Decision Guide

Situation	Best Choice	Why
Need fast random access by index	ArrayList	O(1) get(index) — fastest random access
Frequent insert/delete at beginning/end	ArrayDeque or LinkedList	O(1) addFirst/addLast
Remove duplicates, order does not matter	HashSet	O(1) operations, unique elements
Remove duplicates, preserve insertion order	LinkedHashSet	O(1) + maintains order

Situation	Best Choice	Why
Always want sorted unique elements	TreeSet	Automatically sorted, O(log n)
Key-value lookup, no ordering needed	HashMap	Fastest: O(1) average get/put
Key-value lookup, preserve insertion order	LinkedHashMap	Ordered + O(1) performance
Key-value lookup, keys always sorted	TreeMap	NavigableMap, range queries
Process elements in priority order	PriorityQueue	Min/max heap — priority scheduling
Need both Stack and Queue behaviour	ArrayDeque	Fastest double-ended queue
Thread-safe List operations	CopyOnWriteArrayList	Safe reads, isolated writes
Thread-safe Map operations	ConcurrentHashMap	Segment locking, not full lock
LRU Cache implementation	LinkedHashMap(accessOrder=true)	Built-in eviction support
Store only 1 element immutably	Collections.singletonList/Set	Memory efficient

🎯 Top Interview Questions — Collections Master

❓ Q: What are the differences between Comparable and Comparator?

✅ A: Comparable is in java.lang, implemented by the class itself, method: compareTo(T other), defines natural ordering, only ONE sort order. Comparator is in java.util, implemented externally, method: compare(T o1, T o2), can define MULTIPLE sort orders. Use Comparable for default sort, Comparator for custom/multiple sorts.

❓ Q: What is the difference between fail-fast and fail-safe iterators?

✅ A: Fail-fast: throws ConcurrentModificationException if collection is modified during iteration (ArrayList, HashMap iterators use modCount). Fail-safe: works on a COPY of the collection, no exception (CopyOnWriteArrayList, ConcurrentHashMap iterators). Fail-safe may not reflect latest changes.

❓ Q: How do you make a collection thread-safe without using synchronized collections?

✅ A: Use java.util.concurrent classes: ConcurrentHashMap, CopyOnWriteArrayList, CopyOnWriteArraySet, ConcurrentLinkedQueue, BlockingQueue. These use fine-grained locking (CAS operations) and are much faster than fully synchronized Collections.synchronizedXxx() wrappers.

❓ Q: What is the difference between Enumeration and Iterator?

✅ A: Enumeration is legacy (JDK 1.0), only has hasMoreElements()/nextElement(), cannot remove during iteration, read-only. Iterator is modern (JDK 1.2), has hasNext()/next()/remove(), can safely remove during iteration. Always use Iterator over Enumeration.

📊 SECTION 8 — Data Types: All Differences for Interviews

🔥 MOST IMPORTANT — Primitive vs Non-Primitive

This is the NUMBER ONE most asked data types question in Java interviews. Learn this table cold!

Feature	Primitive Data Types	Non-Primitive (Reference) Types
What are they?	8 basic built-in types in Java	Objects, Arrays, Strings, Classes, Interfaces
Examples	int, long, double, float, char, boolean, byte, short	String, int[], ArrayList, Object, Scanner
Memory location	Value stored directly in STACK memory	Reference (address) in STACK, actual data in HEAP
Default value	Defined: 0, 0.0, false, \u0000	null (points to nothing)
Size	Fixed, known at compile time	Dynamic, determined at runtime
Null allowed?	NO — cannot assign null	YES — reference can be null
Methods available?	None — just value	YES — .length(), .equals(), .toString() etc.
Wrapper class?	Yes (Integer, Double, Boolean etc.)	They ARE objects already
Performance	Faster — direct value access	Slightly slower — requires pointer dereference
Passed to method as	Pass by VALUE (copy of value)	Pass by VALUE of reference (copy of address)
Comparison	== compares values directly	== compares references; use .equals() for content
Autoboxing	Converts to wrapper class automatically	Unboxed to primitive automatically
Use in Collections	Cannot directly (use wrapper class)	Yes — Collections work with objects

🔹 All 8 Primitive Data Types — Complete Details

Type	Size	Wrapper	Default	Min Value	Max Value	Use When
byte	1 byte8-bit	Byte	0	-128	127	File/stream data, pixel values, saving memory in large arrays
short	2 bytes16-bit	Short	0	-32,768	32,767	Rare — audio data, legacy protocols. Use int unless memory critical
int	4 bytes32-bit	Integer	0	-2,147,483,648	2,147,483,647 (~2.1B)	DEFAULT integer choice — age, count, score, loop index, IDs

Type	Size	Wrapper	Default	Min Value	Max Value	Use When
long	8 bytes64-bit	Long	0L	-9.2 × 10^18	9.2 × 10^18	Phone numbers (10 digits), timestamps (System.currentTimeMillis), large counts, DB row IDs
float	4 bytes32-bit	Float	0.0f	±1.4 × 10^-45	±3.4 × 10^38	Graphics, sensor data, GPS where memory > precision. 6-7 sig digits
double	8 bytes64-bit	Double	0.0d	±4.9 × 10^-324	±1.8 × 10^308	DEFAULT decimal — financial calculations (if not BigDecimal), scientific. 15-16 sig digits
char	2 bytes16-bit	Character	\u0000	0 (\u0000)	65535 (\uFFFF)	Single Unicode character — grade (A/B/C), gender (M/F), menu option
boolean	~1 bitJVM dependent	Boolean	false	false (only value)	true (only value)	Flags, conditions, state — isActive, isLoggedIn, isEmpty, hasNext

⚠️ **WARNING:** float has precision issues — float f = 0.1f + 0.2f; is NOT exactly 0.3! For financial applications, use java.math.BigDecimal. double is more precise than float but still has floating-point limitations.

🔹 int vs long vs float vs double — The Most Confused Group

Comparison	int	long	float	double
Type	Integer	Integer	Decimal	Decimal
Size	4 bytes	8 bytes	4 bytes	8 bytes
Precision	Exact	Exact	~6-7 decimal digits	~15-16 decimal digits
Literal	42	42L	3.14f	3.14 or 3.14d
Default?	YES — default int	When int too small	Rarely	YES — default decimal
Example	int age=25;	long ph=9876543210L;	float temp=36.6f;	double pi=3.14159;
When to use	Counts, indices, IDs	Phone, timestamp, big numbers	Memory-critical graphics	Scientific, financial, general decimals

🔹 char vs String — Very Important

Feature	char	String
Type	Primitive	Non-primitive (Object)
Stores	Single character only	Sequence of characters (0 or more)

Feature	char	String
Memory	2 bytes in Stack	Reference in Stack, data in Heap (String Pool)
Default value	\u0000 (null character)	null (reference points to nothing)
Literal syntax	Single quotes: char c = 'A';	Double quotes: String s = "Hello";
Mutable?	Value can be reassigned (new char)	Immutable — once created, content never changes
Comparison	c == 'A' compares value (safe)	s.equals("Hello") — never use == for String content
Methods	None on primitive (Character class)	Many: length(), charAt(), substring(), equals()...
Unicode support	Yes — 2 bytes = 65536 chars	Yes — internally uses char array
Null allowed?	No — char c = null; is error	Yes — String s = null; is valid

🔹 boolean vs Boolean — Tricky Interview Topic

Feature	boolean (primitive)	Boolean (wrapper class)
Type	Primitive	Object
Values	true or false only	true, false, OR null
Memory	~1 bit (JVM decides, often stored as int)	Object in Heap — much more memory
Default in class	false	null
Can be null?	NO	YES — Boolean b = null; is valid
Use in if()	if(flag) — direct	if(boolObj != null && boolObj) — null-check needed!
Use in Map/List	No — must use Boolean	Yes — Collections.frequency(list, Boolean.TRUE)
Autoboxing	boolean → Boolean automatically	Boolean → boolean automatically
NullPointerException risk?	NO	YES — auto-unboxing null Boolean throws NPE!

⚠️ **WARNING:** Boolean b = null; if(b) { ... } throws NullPointerException because auto-unboxing tries to call b.booleanValue() on null. Always null-check wrapper types before unboxing!

🔹 byte vs short vs int — When to Use What

Feature	byte	short	int
Size	1 byte (8-bit)	2 bytes (16-bit)	4 bytes (32-bit)
Range	-128 to 127	-32,768 to 32,767	-2.1 billion to 2.1 billion

Feature	byte	short	int
When to use	Stream/file I/O, pixel data, saving memory in HUGE arrays	Rare — audio processing, legacy code	EVERYTHING ELSE — default integer type
Example	byte[] image = new byte[1024*1024];	short code = 200;	int count = 0;
Arithmetic	Promoted to int in expressions	Promoted to int in expressions	int stays int
Common bug	byte b = 200; // ERROR — out of range	Overflow wraps silently	int + int can overflow — use long!

🔹 String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutability	IMMUTABLE — never changes	MUTABLE — can be changed	MUTABLE — can be changed
Thread safety	Thread-safe (immutable)	NOT thread-safe	Thread-safe (synchronized methods)
Performance	SLOW for concat in loops (creates many objects)	FASTEST (no synchronization overhead)	SLOWER than StringBuilder (sync overhead)
Sync overhead	None	None	Every method is synchronized
Storage	String Pool (literals) or Heap	Heap only	Heap only
When to use	Fixed values, Map keys, general use	Single-threaded string building (loops)	Multi-threaded string building
Key methods	.length(), .charAt(), .substring()	.append(), .insert(), .delete(), .reverse()	Same as StringBuilder
Default size	Length of the string	16 chars (internal buffer)	16 chars (internal buffer)
== comparison	Compares references (NEVER use for content!)	N/A	N/A
Example	"Hello" + "World" = bad in loop	sb.append("Hello").append("World")	sbf.append("Hello")

✅ **TIP:** Interview short answer: "String is immutable, StringBuilder is mutable and fast for single-threaded, StringBuffer is mutable and thread-safe but slower. Use StringBuilder unless multithreaded."

🔹 Primitive vs Wrapper — Auto-boxing Traps!

Primitive	Wrapper	Auto-boxing Trap to Know
int	Integer	Integer cache: Integer.valueOf(-128 to 127) are cached. Integer a=127; Integer b=127; a==b is TRUE. Integer a=200; Integer b=200; a==b is FALSE (new objects)!
long	Long	Same caching as Integer for -128 to 127. Long l = 200L; Long m = 200L; l==m is FALSE
double	Double	No caching for Double. Double.valueOf(1.0) always creates new object. NEVER use == for Double comparison.
boolean	Boolean	Boolean.TRUE and Boolean.FALSE are cached constants. Boolean b = true; b == Boolean.TRUE is true.
char	Character	Character cache: 0 to 127 chars are cached. Above 127 — new object created.

⚠️ **WARNING:** Integer a = 200; Integer b = 200; System.out.println(a == b); prints FALSE! Because 200 is outside the cache range (-128 to 127). This is a VERY COMMON interview trick question. Always use .equals() for wrapper comparison!

🎯 Interview Questions — Data Types

❓ **Q: What is the default value of an int variable in Java?**

✅ A: An INSTANCE variable (field in a class) has default value 0. A LOCAL variable (inside a method) has NO default value — you must initialise it before use. If you try to use an uninitialised local variable, you get a COMPILE ERROR: "variable might not have been initialised".

❓ **Q: Why is String not a primitive type?**

✅ A: String is an object — an instance of java.lang.String class. It has methods (length(), charAt(), etc.), can be null, and is stored in Heap. Primitives have no methods, cannot be null, and are stored in Stack. However, Java gives String special treatment: string literals ("hello") are cached in the String Pool.

❓ **Q: What is the difference between int and Integer?**

✅ A: int is a primitive — 4 bytes in Stack, default 0, no methods, cannot be null, faster. Integer is a Wrapper class — object in Heap, default null, has methods like parseInt(), compareTo(), max(), min(). Auto-boxing converts between them automatically. Collections need Integer (objects), arithmetic prefers int (faster).

❓ **Q: Can we assign long value to int variable?**

✅ A: No — not directly. long is 8 bytes, int is 4 bytes. You need explicit narrowing cast: int i = (int) longValue; — but this may TRUNCATE data if value exceeds int range. Widening (int to long) is automatic: long l = intValue; is fine.

❓ **Q: What happens when int overflows?**

✅ A: Java int silently WRAPS AROUND. int max = Integer.MAX_VALUE; max + 1 = Integer.MIN_VALUE (-2,147,483,648)! No exception, no error — just wraps. To avoid: use long, or check with Math.addExact(a, b) which throws ArithmeticException on overflow.

❓ **Q: What is the difference between float and double?**

✅ A: float: 4 bytes, 6-7 significant decimal digits, literal ends with f (3.14f). double: 8 bytes, 15-16 significant digits, default decimal type (3.14). double is preferred for precision. float is only used when

memory is extremely critical (e.g., large GPU arrays). NEVER use float/double for money — use BigDecimal.

❓ **Q: What is the size of char in Java vs C/C++?**

✅ A: In Java, char is 2 bytes (16-bit Unicode/UTF-16). In C/C++, char is 1 byte (8-bit ASCII). Java char supports full Unicode set (65,536 characters) which is why it is 2 bytes. This is a classic difference question.

❓ **Q: What is type promotion in Java expressions?**

✅ A: In arithmetic expressions, byte and short are automatically promoted to int. If any operand is long, the result is long. If any operand is float, the result is float. If any operand is double, the result is double. Example: byte a=5; byte b=3; byte c = a+b; is COMPILE ERROR — a+b becomes int!

❓ **Q: What is the difference between == and .equals() for Integer objects?**

✅ A: Integer caches values -128 to 127. For values in this range, == works (same cached object). For values outside this range, == returns false even if values are equal (different objects). ALWAYS use .equals() for Integer comparison — never == . This is one of the most common Java interview traps!

🔹 Quick Revision — Type Casting in Java

```
// WIDENING (automatic) — smaller to larger type — no data loss
byte  → short → int → long → float → double

byte b = 50;
int i = b;           // automatic widening — OK
double d = i;        // automatic widening — OK

// NARROWING (explicit) — larger to smaller — POSSIBLE data loss
double → float → long → int → short → byte

double pi = 3.14159;
int truncated = (int) pi;         // 3 — decimal part LOST
byte small = (byte) 130;         // -126 — OVERFLOW, wraps around!

// Char and int casting (very common in interviews)
char c = 'A';
int ascii = c;                   // 65  — char to int (widening)
char back = (char)(ascii + 1);   // 'B' — int to char (narrowing)

// String to number
int n = Integer.parseInt("42");
double d2 = Double.parseDouble("3.14");

// Number to String
String s1 = String.valueOf(42);
String s2 = Integer.toString(42);
String s3 = "" + 42;              // concatenation trick
```

🏆 FINAL MASTER TABLE — All Data Types Side by Side

Type	Category	Size	Default	Range / Values	Wrapper	Real Use Case
byte	Primitive integer	1 byte	0	-128 to 127	Byte	Image pixels, I/O streams

Type	Category	Size	Default	Range / Values	Wrapper	Real Use Case
short	Primitive integer	2 bytes	0	-32768 to 32767	Short	Audio, legacy APIs
int	Primitive integer	4 bytes	0	-2.1B to 2.1B	Integer	Age, score, count, index
long	Primitive integer	8 bytes	0L	-9.2×10^18 to 9.2×10^18	Long	Phone, timestamp, big IDs
float	Primitive decimal	4 bytes	0.0f	±3.4×10^38 (6-7 dig)	Float	Graphics, sensors
double	Primitive decimal	8 bytes	0.0d	±1.8×10^308 (15-16 dig)	Double	Money*, science, general decimal
char	Primitive char	2 bytes	\u0000	0 to 65535 (Unicode)	Character	Grade, symbol, single letter
boolean	Primitive logical	~1 bit	false	true / false only	Boolean	Flags, conditions
String	Non-primitive	Dynamic (Heap)	null	Any text (0+ chars)	Is object	Names, messages, keys
int[]	Non-primitive	Dynamic (Heap)	null	Array of ints	Integer[]	Lists of numbers
Integer	Wrapper object	~16 bytes	null	-2.1B to 2.1B (+ null)	itself	Collections, nullable int
BigDecimal	Non-primitive	Dynamic	null	Unlimited precision	itself	FINANCIAL calculations

🔥 **REMEMBER:** * Never use float or double for financial calculations! 0.1 + 0.2 in floating point is NOT exactly 0.3. Use java.math.BigDecimal for any money-related code.